Rolling Hash を法なしで理解する

—— 1〜9 の素朴な例から始める ——

$\Sigma = \{a, b, c, d, e, f, g, h, i\}$

$\varphi(a) = 1,\ \varphi(b) = 2, \cdots, \varphi(i) = 9$

$r = 10$

$S = S_0 S_1 \cdots S_{n-1}$ （文字列）

$$H(S) = \varphi(S_0) r^0 + \varphi(S_1) r^1 + \cdots + \varphi(S_{n-1}) r^{n-1}$$

以後、数字に直した S_i をそのまま S_i と書くこともある

直感 「逆向きに読む十進法」

→ 先頭の文字 S_0 が一の位（1の位）になる。

例

head → 4158 （$h=8,\ e=5,\ a=1,\ d=4$）

badge → 54127 （$b=2,\ a=1,\ d=4,\ g=7,\ e=5$）

法を取らない限り、これは普通の位取り記数法そのもの

部分文字列の一致判定を $O(m+n)$ でやる

— ※ ここでは「部分列」ではなく「部分文字列」を考える —

問題 長さ n の文字列 S の中に、長さ m の文字列 T が連続して現れるか？

素朴法 素朴には各位置で m 文字比較 $\rightarrow O(mn)$

例 $S = badge$ （$m = 3$ の窓）

$bad \rightarrow H = 412$
$adg \rightarrow H = 741$
$dge \rightarrow H = 574$

更新の例： 「bad」から「adg」へ

$H(bad) = 412$
先頭 $b = 2$ を捨てる： $(412 - 2) / 10 = 41$
末尾 $g = 7$ を足す： $41 + 7 \cdot 10^2 = 741 = H(adg)$

一般の更新式

$$H(next) = \frac{H(cur) - \varphi(\text{先頭文字})}{r} + r^{m-1}\varphi(\text{新しい末尾文字})$$

計算量

- $H(T)$ を計算： $O(m)$
- 最初の窓を計算： $O(m)$
- 以後は 1 回ずらすごとに $O(1)$

$\rightarrow$ 合計 $O(m+n)$

更新式の一般形

$S = S_0 S_1 \cdots S_{n-1}$ | 長さ m の窓　$W_i = S_i S_{i+1} \cdots S_{i+m-1}$

$$H(W_i) = \varphi(S_i) + \varphi(S_{i+1}) r + \cdots + \varphi(S_{i+m-1}) r^{m-1}$$

更新の導出

① 先頭 S_i を落とす　$\longrightarrow$　$H(W_i) - \varphi(S_i)$

② r で割る　$\longrightarrow$　$\dfrac{H(W_i) - \varphi(S_i)}{r}$

③ 新しい末尾 S_{i+m} を足す　$\longrightarrow$　$+ r^{m-1} \varphi(S_{i+m})$

更新式

$$H(W_{i+1}) = \frac{H(W_i) - \varphi(S_i)}{r} + r^{m-1} \varphi(S_{i+m})$$

同値な式

$$r H(W_{i+1}) - H(W_i) = r^m \varphi(S_{i+m}) - \varphi(S_i)$$

解釈　共通部分はそのまま、出る文字と入る文字だけを調整する

ただし法を取らないと数がどんどん大きくなる $\longrightarrow$ 次に mod を入れたくなる

連結に対するふるまい → 自然な抽象化

A, B を文字列とする | $H(AB) = H(A) + r^{|A|} H(B)$ | B の桁は、A の長さだけ左へずれる

例

$A = ba, \quad B = dge$

$H(ba) = 12, \quad H(dge) = 574$ （1〜9 の設定, $r = 10$）

$H(badge) = 12 + 10^2 \cdot 574 = 57412$

直感 長さも一緒に持てば、式がきれいになる

$F(S) = (H(S), |S|)$ | $(h, n) \bigstar (k, m) = (h + r^n k, n + m)$

$$F(AB) = F(A) \bigstar F(B)$$

つまり、Rolling Hash は「連結」と相性がよい | **付録**：これはモノイド準同型・半直積として見られる

付録A：モノイドって何？

— まずは、モノイド（monoid）の超かんたんな定義から！ —

集合 M と演算 $*$ があって

(1) 結合律　$(a*b)*c = a*(b*c)$

(2) 単位元 e がある　$e*a = a*e = a$

→ これをモノイドという

例1　文字列全体 Σ^* と連結

- 単位元は空文字 ε

例2　自然数の組 (h,n) と ★

- $(h,n) \bigstar (k,m) = (h + r^n k,\ n+m)$
- 単位元は $(0,0)$

重要なつながり

$F(S) = (H(S), |S|)$ は、文字列の連結を ★ にうつす

$$F(AB) = F(A) \bigstar F(B), \quad F(\varepsilon) = (0,0)$$

まとめ　こういう「演算を保つ写像」をモノイド準同型という

付録B：半直積と、なぜこれが出てくるのか

演算

$$(h, n) \star (k, m) = (h + r^n k,\ n + m)$$

→ 長さ n が、後ろの hash k を r^n 倍している

→ つまり「長さ」が「hash 値」に作用している

作用

$$\rho_n(k) = r^n k$$
$$\rho_{n+m} = \rho_n \circ \rho_m$$

直積ではなく、片方がもう片方に作用してできる積 → 半直積

記法

$$(\mathbb{N}, +) \ltimes_\rho (\mathbb{N}, +)$$

左右ともに加法モノイド $((\mathbb{N}, +))$

（参考） $h' = \dfrac{h - a}{r} + r^{m-1} b$ は、固定した a, b, m ではアフィン変換 $h' = \alpha h + \beta$

Rolling Hash の公式は、偶然のテクニックではなく、
連結の代数的構造を使っている